Incremental Search-Based Allocation of Autonomous Robots for Goods Delivery

Paolo Arcaini
National Institute of Informatics
Tokyo, Japan
arcaini@nii.ac.jp

Ezequiel Castellano
National Institute of Informatics
Tokyo, Japan
ecastellano@nii.ac.jp

Fuyuki Ishikawa
National Institute of Informatics
Tokyo, Japan
f-ishikawa@nii.ac.jp

Hirokazu Kawamoto
Panasonic Holdings Corporation
Tokyo, Japan
kawamoto.hirokazu@jp.panasonic.com

Kaoru Sawai
Panasonic System Networks R&D Lab. Co., Ltd.
Miyagi, Japan
sawai.kaoru@jp.panasonic.com

Eiichi Muramoto
Panasonic Holdings Corporation
Tokyo, Japan
muramoto.eiichi@jp.panasonic.com

***Abstract*—Autonomous robots can solve different issues of delivery services, by guaranteeing less traffic congestion, less pollution, and lower operational costs. Designing such type of delivery system based on autonomous robots requires the collaboration of different stakeholders, having different concerns: the store utilising the delivery service that is interested in costs and customer satisfaction, the municipality where the service is operated that is interested in the safety of the service, and the robotic company providing the service that is interested in all previous concerns. Our industrial partner from the robotic domain is designing this type of service in a smart town, and using a simulator for assessing different configurations providing different levels of performance. Since manually designing the configurations is time consuming for engineers, in this paper, we propose a search-based approach (`All`) that is able to explore the space of service configurations and find the optimal ones that show the trade-off existing among the different concerns, so that stakeholders can make an informed decision. Since assessing one configuration requires to simulate the service multiple times over different types of customer requests, the approach suffers from scalability issues. Therefore, we propose two improvements of the approach that reduce the number of required simulations (`IncrSim`), and the duration of the simulation (`IncrTime`). Experiments on different settings show that `IncrSim` and `IncrTime` can find results as good as those of `All` in less time, and better than versions of `All` executed for the same budget.**

***Index Terms*—autonomous robots, goods delivery, search-based allocation, approximate fitness**

I. Introduction

In recent years, services for delivery of food, groceries, and other types of goods have become widespread. However, this creates many issues in the last-mile delivery, i.e., the final delivery of the good from the store (supermarket, restaurant, grocery store, etc.) to the house of the customer; among these, traffic congestion, pollution, and operational costs.

An effective solution to solve the above mentioned issues is the adoption of *autonomous delivery robots* [16], that can operate at much lower cost, with much less environmental impact, and reducing the traffic congestion.

In this context, our industrial partner Panasonic is experimenting with an *automatic delivery service* performed by autonomous robots in the Fujisawa Sustainable Smart Town, Japan. Specifically, customers of the town can order goods through an application to a local supermarket. The requests are collected by an operation center that schedules a fleet of autonomous robots to deliver the ordered goods. When assigned to a customer request, a robot drives to the supermarket, picks the good up, and delivers it to the customer. Although they are autonomous, robots are monitored remotely by some human operators that can intervene whenever some unexpected event happens, e.g., a too-close interaction with a pedestrian.

Currently, the automatic delivery service is in the experimentation phase, and the three main stakeholders are assessing its costs and benefits. The municipality where the service is operated is assessing the quality of the service provided to its citizens and, at the same time, the level of guaranteed safety. The supermarket using the service is considering the cost of using the service, and the amount of requests that can be handled. Panasonic, as provider of the service, is interested in all the previously mentioned concerns.

Assessing the automatic delivery service requires to try several different *configurations* in terms of *number of robots*, their *operating hours*, and their *speed*. Indeed, having more robots working for more hours allows to serve more requests, but also increases the overall costs. Moreover, increasing the speed of the robots allows to increase the rate of served requests, but also increases the probability of dangerous interactions that require the intervention of the remote human operators.

Since trying all the configurations in the real world is not feasible, the company is experimenting with a simulator that allows to specify as input: a set of customer requests,

P. Arcaini and F. Ishikawa are supported by ERATO HASUO Metamathematics for Systems Design Project (No. JPMJER1603), JST, Funding Reference number 10.13039/501100009024 ERATO; and by Engineerable AI Techniques for Practical Applications of High-Quality Machine Learning-based Systems Project (Grant Number JPMJMI20B8), JST-Mirai.

the operating times of a set of robots, and their speed. The simulator simulates the scheduling of the different robots for serving the requests, and the actual delivery process; as output, it reports simulation data in terms of requests that have been actually served, delivery rate of each robot, and the number of interventions of the human operators.

Proposed approach. At the beginning of the experimentation, the engineers of Panasonic started manually designing different configurations to be evaluated with the simulator. However, due to the large configuration space, such process is too costly and does not allow to properly detect the trade-off existing among the different concerns. Therefore, in this paper, we propose a search-based approach (called `All`) that searches for the optimal configuration of the service (the individual of the search), i.e., number of robots, their operating hours, and their speed. The approach has three objectives: (i) maximising the customer requests that are served; (ii) maximising the utilisation of the used robots; (iii) minimising the number of human interventions.

To assess a possible configuration, the approach simulates it using different sets of customer requests SETSREQS $= \{Reqs_1, \ldots, Reqs_{NumSRs}\}$, that are considered by the company representative of possible future patterns of requests; indeed, the same configuration of the service should be optimal, on average, for the different possible usages of the service.

Running the simulation for each set of requests $Reqs_i$ can take up to minutes depending on the simulated time. Therefore, since each individual must be simulated for $NumSRs$ sets of requests, the approach suffers from some scalability issues. To tackle these issues, we propose two *incremental* versions of the approach that allow to obtain more scalability:

- `IncrSim`: this version of the approach does not simulate all the individuals on all the sets of customer requests in SETSREQS; specifically, earlier generations of the search are assessed using fewer sets of customer requests, while later generations are assessed with an increasing number of sets. The intuition is that individuals in earlier generations are more diverse in terms of performance (i.e., values of the fitness functions) and so also an imprecise assessment should be enough to discriminate between good and bad solutions; individuals in later generations, instead, are much closer in terms of performance, and so we need a more precise assessment to evaluate them;
- `IncrTime`: this version of the approach does not simulate all the individuals for the whole interval in which the service is operating; specifically, individuals of earlier generations are only assessed on a sub-interval, that is increased with the increment of the generations. Similarly to the `IncrSim` approach, the intuition is that, in earlier generations, an imprecise assessment is enough to discriminate among solutions, while the assessment must become more precise in later generations.

We conducted experiments using different number of generations, different number of sets of customer requests in SETSREQS, different number of requests in each set $Reqs_i$, and different duration of the delivery service. Experimental results show that the proposed incremental approaches `IncrSim` and `IncrTime` are as good as the costly `All` approach, so showing that we can achieve scalability without sacrificing the optimality. Moreover, `IncrSim` and `IncrTime` are better than versions of the `All` approach that are executed for the same budget in terms of number of simulations and number of simulated hours; this shows that the allocation of the budget performed by `IncrSim` and `IncrTime` is effective.

Paper structure. Sect. II introduces the optimal allocation problem. Then, Sect. III presents the search-based approach we propose, and the two incremental versions. Sect. IV describes the design of the experiments we conducted to assess the approach, and Sect. V discusses experimental results. Sect. VI discusses some threats that may affect the validity of the approach and how we mitigated them. Finally, Sect. VII reviews some related work, and Sect. VIII concludes the paper.

II. PROBLEM DESCRIPTION

In this paper, we consider the system provided by our industrial partner from the robotic domain. It consists of an *automatic delivery service* in which users (called *customers* in the following) of a residential area order goods from a local supermarket, and these are delivered by autonomous robots. Specifically, the system works as follows:

- customers make *delivery requests* through an application (called *customer requests* in the following); a request $req = \{g, l, [s_g, e_g]\}$ consists in the order of goods g, to be delivered at location l, in the desired time interval $[s_g, e_g]$;
- requests are received by an automatic operation center OC; for each request $\{g, l, [s_g, e_g]\}$, OC schedules a specific robot $r \in Robs = \{r_1, \ldots, r_{nr}\}$ to serve the request. Different algorithms are available to do the scheduling [12], [14], [5]; in this work, we assume that a specific algorithm is given; note that the target of our investigation is not the scheduling algorithm, but the allocation of the robots that are scheduled;
- if a robot is assigned to the delivery of a good, it goes to the central deposit, it picks the good up, and then drives to the house of the customer;
- while driving, the robot is remotely monitored by a *human operator*; if the robot approaches a dangerous situation (e.g., too close to a pedestrian), the human operator overrides the robot operation and automatically stops it or performs some evading manoeuvre. The probability of human intervention directly depends on the maximum robot speed rs (that is the same for all robots in $Robs$): the higher the speed is, the higher the probability is that the robot incurs in dangerous situations that require human intervention. Requirements specify that each human operator can supervise at most two robots.

During the day, the automatic delivery service is available during a *service interval* $SI = [SI_s, SI_e]$; service intervals that are being considered by our industry partner are [7:00,

12:00] (i.e., only in the morning) and [7:00, 19:00] (i.e., the whole day). We identify with D^{SI} the *service interval duration* in number of hours[1], i.e., $D^{SI} = SI_e - SI_s$. Each robot can operate for the whole service interval or only for a fraction of it; for simplicity, we consider one hour as the minimal granularity of robot operation. We will identify with $Reqs$ the set of customer requests received for a service interval.

A. Optimal allocation problem

Our industry partner is in the process of experimenting the automatic delivery service and discussing the requirements with the *stakeholders*: (i) the *business management* of the company itself, (ii) the *supermarkets* that will use the service, (iii) the *municipality* where the system will operate.

The problem that our industry partner is facing is how to decide: (i) *how many robots* to have available for the service, (ii) their *operating times*, i.e., when to make them available during the day, and (iii) their *maximum speed*. Different considerations must be taken into account in this process:

- one of the main goals of the delivery service is to serve all the requests in their specified interval $[s_g, e_g]$; note that either a request is served in $[s_g, e_g]$, or it is not served at all (i.e., it cannot be served with a delay);
- delivery robots must be actually built, and each individual robot has a non-negligible cost. So, the creation of a robot must be planned in advance and must be motivated by well-founded business reasons, and the company must decide in advance how many robots to use for the service;
- increasing the speed rs of the robots increases the number of requests that can be served by each individual robot; however, at the same time, it increases the probability that human intervention is needed. If the probability of human intervention increases, more human operators are needed, so increasing the cost of human resources;
- the delivery service cannot be optimised on a daily basis. First of all, all the requests of the users may not be known in advance; moreover, building and distributing robots takes some time. So, the industrial partner wants to decide in advance how many robots to allocate. To do this, some forecasted patterns of customer requests are used in this decision.

Given the previous concerns, our partner wants to optimise the system along the following directions:

- **Optimisation goal 1**: maximising the number of served requests;
- **Optimisation goal 2**: minimising the number of robots, while maximising their usage;
- **Optimisation goal 3**: minimising the number of human interventions.

This is a multi-objective problem whose solution is a set of non-dominated solutions (i.e., a Pareto front) that show the trade-off among the different objectives. Having the documentation of such trade-off is very important for our industrial partner, as different stakeholders are involved in deciding how many robots to allocate and when to allocate them, and they have different concerns in these decisions: (i) supermarkets that employ the delivery service are concerned by the delivery rate and by the cost of robots, because they want to maximise the satisfaction of their clients but, at the same time, minimise the cost of the service; (ii) Panasonic is concerned by all the objectives; (iii) the municipality in which the robots are going to be delivered, is concerned by the number of human interventions, that identify situations in which the robot interacts with humans or elements of the street; indeed, although these situations do not constitute an immediate danger, a high number of such episodes could diminish the social acceptance of the system and so prevent its adoption.

III. Incremental search-based allocation

In order to reason on the different possible solutions for the automatic delivery service, Panasonic developed a simulator that allows to specify a *configuration* in terms of number of robots, their operating times and their speed, and a set of customer requests; given a configuration, the simulator simulates the whole delivery system reporting which customers requests have been delivered, and which robot delivered each request. In the current practice, engineers of Panasonic are manually experimenting with different configurations to try to solve the optimal allocation problem introduced in Sect. II-A. However, this process is difficult and time-consuming for engineers.

Therefore, in this paper, we propose a search-based approach to solve the optimal allocation problem. At a high level, the approach consists in trying to find the best robot allocation by assessing it against different sets of customer requests $\text{SETSREQS} = \{Reqs_1, \ldots, Reqs_{NumSRs}\}$, where $Reqs_i = \{req^i_1, \ldots, req^i_{i_n}\}$; each $Reqs_i$ is a set of customer requests that can occur during a service interval SI. These sets are taken as representative of future sets of requests, that could happen, for example, in different days of the week and/or different seasons. Such sets of requests can be obtained either from historical data, or can be forecasted data; since the company does not have historical data at this stage of development, in this work we use forecasted data. The allocation should provide, on average, good results on all the sets of customer requests.

The industry partner is considering two *allocation settings*:

- *Partial allocation* (`PartAllo`): in this setting, a robot is allocated for a sub-interval of the whole service interval;
- *Full allocation* (`FullAllo`): in this setting, all the robots are allocated for the whole service interval.[2]

In the following, we propose a search-based approach (called `All`) for solving the allocation problem in the two settings. The main issue of the allocation problem is that each possible solution (i.e., allocation of the robots with their speed) must be evaluated for $NumSRs$ sets of customer requests, and each evaluation requires the simulation of the system with the

[1]Note that, for the sake of simplicity, we assume that service intervals start and end on the hour.

[2]Note that, although `FullAllo` can be considered as a special case of `PartAllo`, it is more convenient to consider the two settings separately.

simulator that can take up to minutes, and increases with the number of simulated hours (i.e., the service interval duration D^{SI}). So, a scalability issue affects the applicability of the search-based approach.

Therefore, we also present two improvements of the approach that reduce the number of simulations (approach `IncrSim`) and the length of the simulations (approach `IncrTime`). While `IncrSim` can be applied to the search for both settings `PartAllo` and `FullAllo`, `IncrTime` can only be applied to the search for setting `FullAllo`.

We present in Sect. III-A the approach for solving the problem in the partial allocation setting, with the improvement `IncrSim`. Then, in Sect. III-B, we present the approach for solving the problem in the full allocation setting, with the improvements `IncrSim` and `IncrTime`.

A. Partial allocation (`PartAllo`) setting

We here describe the approach for finding the best allocation in the `PartAllo` setting. Concretely, we need to find (i) how many robots to use during the service interval, (ii) when to use each of them, and (iii) their speed. We first propose an approach (`All`) that always simulates all the requests (see Sect. III-A1); then, we propose a modification of the approach (`IncrSim`) that avoids running all the simulations all the times (see Sect. III-A2).

1) `All` *approach:* In the following, we describe the proposed search-based approach. We first introduce the definition of individual, and then of objective functions.

a) Individual of the search problem: We assume that, at most, MR robots can be allocated for the problem, but possibly also less.

An individual of the search determines the working hours of each robot and the maximum speed of all the robots. Specifically, the *search variables* $\overline{x}$ are:

$$\overline{x} = [x^1_{rs}, x^1_{rwh}, \ldots, x^{MR}_{rs}, x^{MR}_{rwh}, x_{speed}]$$

where:

- x^i_{rs} is an integer variable that identifies the starting time of operation of robot r_i, with $x^i_{rs} \in \{SI_s, \ldots, SI_e - 1\}$, and $i \in \{1, \ldots, MR\}$;
- x^i_{rwh} is an integer variable that identifies the number of working hours of robot r_i, with $x^i_{rwh} \in \{0, \ldots, SI_e - SI_s\}$, and $i \in \{1, \ldots, MR\}$. If $x^i_{rwh} = 0$, it means that robot r_i is not selected; in this way, we decide the number of robots. The ending operation time of the robot is computed as $x^i_{re} = \min(x^i_{rs} + x^i_{rwh}, SI_e)$;
- x_{speed} is the maximum speed of all the robots.

An *individual* $\overline{v}$ is a concrete assignment to variables $\overline{x}$:

$$\overline{v} = [v^1_{rs}, v^1_{rwh}, \ldots, v^{MR}_{rs}, v^{MR}_{rwh}, v_{speed}]$$

Given an individual $\overline{v}$ and a set of customer requests $Reqs_i$, we can simulate the system to obtain its performance:[3]

$$\texttt{sim}(\overline{v}, Reqs_i) = \left\langle SR^{\overline{v}}_i, UR^{\overline{v}}_i, OI^{\overline{v}}_i \right\rangle$$

[3]Note that the simulator provides other simulation data that, however, is not relevant for this work and so we avoid reporting it.

where:

- $SR^{\overline{v}}_i$ is the percentage of customer requests in $Reqs_i$ that have been served per hour (the *delivery rate* per hour); note that, at this level of the service design, our industry partner is also interested in solutions that do not satisfy all the requests. Indeed, according to the results of our study, they may decide to limit the maximum number of customer requests, or inform the customers when the delivery is not guaranteed in the chosen delivery interval;
- $UR^{\overline{v}}_i = \{ur^{\overline{v}}_1, \ldots, ur^{\overline{v}}_{nr}\}$ reports, for each selected robot r_i, its *utilisation rate* ur_i, i.e., the average utilisation per hour; nr ($nr \leq MR$) is the number of selected robots;
- $OI^{\overline{v}}_i$ reports the average number of interventions of the human operator per hour.

Given all the sets of customer requests $\textsc{SetsReqs} = \{Reqs_1, \ldots, Reqs_{NumSRs}\}$, the whole simulation results are:

$$SimRes(\overline{v}, \textsc{SetsReqs}) = \bigcup_{Reqs_i \in \textsc{SetsReqs}} \{\texttt{sim}(\overline{v}, Reqs_i)\}$$

For a given individual $\overline{v}$, we can also determine how many robots have been used, i.e.,

$$SelRob(\overline{v}) = |\{i \in \{1, \ldots, MR\} \mid v^i_{rwh} \neq 0\}|$$

So, we set $nr = SelRob(\overline{v})$.

b) Objective functions of the search problem: The objective functions formalise the general optimisation goals stated in Sect. II-A.

The first objective consists in maximising the average delivery rate across the $NumSRs$ sets of requests $\textsc{SetsReqs}$; the fitness function that must be maximised is:

$$f_{delRate}(\overline{v}) = \frac{1}{NumSRs} \sum_{\substack{\langle SR^{\overline{v}}_i, UR^{\overline{v}}_i, OI^{\overline{v}}_i \rangle \in \\ SimRes(\overline{v}, \textsc{SetsReqs})}} SR^{\overline{v}}_i \quad (1)$$

The second objective consists in maximising the average utilisation of the robots across the $NumSRs$ sets of requests; the fitness function that must be maximised is:

$$f_{util}(\overline{v}) = \frac{1}{NumSRs} \sum_{\substack{\langle SR^{\overline{v}}_i, UR^{\overline{v}}_i, OI^{\overline{v}}_i \rangle \in \\ SimRes(\overline{v}, \textsc{SetsReqs})}} \sum_{ur \in UR^{\overline{v}}_i} \frac{ur}{SelRob(\overline{v})} \quad (2)$$

Finally, the third objective consists in minimising the average number of interventions of the human operators across the $NumSRs$ sets of requests; the fitness function that must be minimised is:

$$f_{humOp}(\overline{v}) = \frac{1}{NumSRs} \sum_{\substack{\langle SR^{\overline{v}}_i, UR^{\overline{v}}_i, OI^{\overline{v}}_i \rangle \in \\ SimRes(\overline{v}, \textsc{SetsReqs})}} OI^{\overline{v}}_i \quad (3)$$

2) `IncrSim` *– Incremental Simulations Search:* Simulating the delivery service with the simulator is expensive. Therefore, the fitness evaluation is the bottleneck of the `All` approach, as it requires to perform $NumSRs$ simulations (i.e., the number of sets of requests in $\textsc{SetsReqs}$).

TABLE I: `IncrSim` – Example for $NumGens$ = 10, $NumSRs$ = 5

g	$NumSRs_g$	SETSREQS_g
1-2	1	$\{Reqs_1\}$
3-4	2	$\{Reqs_1, Reqs_2\}$
5-6	3	$\{Reqs_1, Reqs_2, Reqs_3\}$
7-8	4	$\{Reqs_1, Reqs_2, Reqs_3, Reqs_4\}$
9-10	5	$\{Reqs_1, Reqs_2, Reqs_3, Reqs_4, Reqs_5\}$

In this section, we describe the `IncrSim` approach, that tries to improve the performance of the `All` approach by not simulating an individual on all the sets of customer requests, but only on some of them. Specifically, we assume that early generations of the search do not need to evaluate the individuals on all the sets of customer requests, but even a reduced number of them should be sufficient to discriminate good from bad individuals. Indeed, at the beginning of the search, individuals are much more diverse in terms of performance; so, the imprecision in the fitness due to the reduced number of simulations (w.r.t. the assessment using all the sets) should not affect the ranking of the solutions. Instead, as the search progresses in later generations, the individuals are usually better and close to each other in terms of performance; therefore, for these generations, individuals should be assessed on more (or all) sets of customer requests to get more trustworthy results.

Technically, we capture the previous intuition as follows. Given the total number of generations $NumGens$, the number of sets of customer requests $NumSRs_g$ that are considered at generation $g \in \{1, \ldots, NumGens\}$ is:

$$NumSRs_g = \left\lceil \frac{g}{NumGens} \times NumSRs \right\rceil$$

So, at generation g, we use the set of requests $\text{SETSREQS}_g = \{Reqs_1, \ldots, Reqs_{NumSRs_g}\}$. SETSREQS_g and $NumSRs_g$ are used in Eqs. 1, 2, and 3.

Table I reports an example of the sets of requests considered by the approach when run for 10 generations, using a total of 5 sets of customer requests.

B. Full allocation (`FullAllo`) setting

We here describe the approach for finding the best allocation in the `FullAllo` setting. Concretely, we need to find how many robots to use during the service interval SI, and their speed. Recall that, in this setting, all the selected robots are executed for the whole service interval. We first describe how the `All` approach can be applied also in this setting (see Sect. III-B1); then, we describe another type of incremental search that is applicable only in this setting (see Sect. III-B2).

1) `All` *approach:* The approach is very similar to the one for the `PartAllo` setting (see Sect. III-A1). The only differences are:

- the *search variables* $\overline{x}$ are:

$$\overline{x} = [x_{nr}, x_{speed}]$$

where $x_{nr} \in \{1, \ldots, MR\}$ is an integer variable specifying the number of robots, and x_{speed} their speed. An individual of the search is then

$$\overline{v} = [v_{nr}, v_{speed}]$$

- the objective functions $f_{delRate}$ and f_{humOp} are the same as those in Eqs. 1 and 3, while f_{util} is slightly modified as follows:

$$f_{util}(\overline{v}) = \frac{1}{NumSRs} \sum_{\substack{\langle SR_i^{\overline{v}}, UR_i^{\overline{v}}, OI_i^{\overline{v}} \rangle \in \\ SimRes(\overline{v}, \text{SETSREQS})}} \sum_{ur \in UR_i^{\overline{v}}} \frac{ur}{v_{nr}} \quad (4)$$

For this approach, the optimisation `IncrSim` described in Sect. III-A2 can be still applied in the exact same way.

2) `IncrTime` – *Incremental Time Search:* For this setting, we can perform also a different optimisation that does not always run a simulation for the whole service interval $SI = [SI_s,\ SI_e]$, but for a portion of it $[SI_s,\ SI_e^i]$, with $SI_e^i < SI_e$. This optimisation starts from the observation that the robots are allocated to different requests multiple times during the service interval SI; so, we may think that the allocation problem for the whole service interval can be split, roughly, in subsequent allocation problems for shorter intervals.

So, we propose this incremental approach. In the first generations of the search, we try to find the best allocation for the initial sub-interval; this allocation should already be sufficiently good, but, of course, it does not guarantee to be optimal for a longer period. So, in subsequent generations, we prolong the interval over which to evaluate the new individuals. We continue extending in this way till we obtain the whole service interval.

Technically, we capture the previous intuition as follows. Given the total number of generations $NumGens$, the duration of the simulation at generation $g \in \{1, \ldots, NumGens\}$ is:

$$D_g^{SI} = \left\lceil \frac{g}{NumGens} \times D^{SI} \right\rceil$$

The service interval is adjusted accordingly as follows:

$$SI_g = [SI_s, SI_s + D_g^{SI}]$$

So, at generation g, the simulations are run for the service interval SI_g.

Note that the fitness values are comparable across generations having different service intervals. Indeed, all the metrics are always provided as average per hour, and so the magnitudes of the fitness functions are always the same.

Table II reports an example of the sets of requests considered by the approach when run for 10 generations, using a whole service interval of 5 hours.

IV. Experiment design

In this section, we describe the design of the experiments we conducted to assess the proposed incremental approaches. We first describe the compared approaches in Sect. IV-A. Then, in Sect. IV-B, we describe how we run the experiments and

TABLE II: IncrTime – Example for $NumGens$ = 10, SI = [7:00, 12:00] (D^{SI} = 5)

g	D_g^{SI}	SI_g
1-2	1	[7:00, 8:00]
3-4	2	[7:00, 9:00]
5-6	3	[7:00, 10:00]
7-8	4	[7:00, 11:00]
9-10	5	[7:00, 12:00]

how we assess the different approaches. Finally, in Sect. IV-C, we present the research questions we use to analyse the experimental results. All the code and experimental results are reported online [2]; note that, for IP protection, we cannot provide the industrial simulator.

A. Compared approaches

We will experiment the proposed incremental approaches IncrSim and IncrTime with different settings. Maximum number of generations $NumGens$: 10, 20. Service interval duration D^{SI}: 5 and 12 hours. Number of sets of customer requests $NumSRs$: 5 and 10. Numbers of requests $NumReqs$ per hour in a customer requests set: 50 and 100. So, in total, we have 16 settings of the experiments. In all the settings, the population size has been set to 12.

For each setting, we set the maximum number of robots $MR = 10$ (which determines the number of variables in the PartAllo setting). The maximum possible speed of the robots is set to 20 km/h; in order to search for speeds with one decimal digit but using an integer variable x_{speed}, we search in the set $\{1, \ldots, 100\}$ and divide the obtained value v_{speed} by 10 to get the actual speed of the robots.

For each setting, we will compare IncrSim and IncrTime with the baseline approach All that always simulates an individual for all the sets of customer requests SETSREQS for the whole service interval SI. The aim of this comparison is to check whether the proposed approaches, although they use a lower budget (i.e., less simulations and less simulated hours), obtain results as good as those of the costly All baseline.

We will further compare IncrSim and IncrTime with versions of the All approach (called $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ and $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$) that are given the same (or slightly higher) budget. Specifically, $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ is given the same budget of IncrSim in terms of number of simulations. When executed for 10 generations, IncrSim uses a total of 360 simulations when $NumSRs = 5$, and 660 simulations when $NumSRs = 10$; to have comparable budgets, $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ is executed for 6 generations for $NumSRs = 5$ (that use exactly 360 simulations), and for 6 generations for $NumSRs = 10$ (that use 720 simulations). The execution of IncrSim for 20 generations uses 720 simulations for $NumSRs = 5$, and 1320 simulations for $NumSRs = 10$; it is compared with $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ executed for 12 and 11 generations in the two settings, that use exactly 720 and 1320 simulations.

IncrTime runs the simulation for an incremental number of hours. We compare it with $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$ that uses the same (or similar) number of hours; note that the simulation time is linear with the number of simulated hours. So, defining the budget in terms of simulated hours is fair. When IncrTime is executed for 10 generations, it uses 360 hours (for each set of customer requests) in the setting in which $D^{SI} = 5$, and it uses 840 hours in the setting in which $D^{SI} = 12$; then, $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$ is run for 6 generations in both settings, which correspond to, respectively, 360 and 864 hours. When IncrTime is executed for 20 generations, it uses 720 and 1608 hours in the two settings of D^{SI}; then, $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$ is run for 12 generations in both settings, which correspond to, respectively, 720 and 1728 hours.

TABLE III: Budget allocation

(a) Number of total simulations. For $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$, we also report in parentheses the number of corresponding generations

$NumSRs$	5	5	10	10
$NumGens$	10	20	10	20
All	600	1200	1200	2400
IncrSim	360	720	660	1320
$\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$	360 (6)	720 (12)	720 (6)	1320 (11)

(b) Number of simulated hours (for one set of customer requests). For $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$, we also report the number of corresponding generations

D^{SI} (hours)	5	5	12	12
$NumGens$	10	20	10	20
All	600	1200	1440	2880
IncrTime	360	720	840	1608
$\texttt{All}_{\texttt{SB}}^{\texttt{time}}$	360 (6)	720 (12)	864 (6)	1728 (12)

Summary of the budget allocated to the different approaches for the different configurations is shown in Table III. Table IIIa reports the budget in terms of total number of simulations for All, IncrSim, and $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$; since $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ is run for less generations (in order to have a budget comparable with the one of IncrSim), for it we also report in parentheses the number of generations. Table IIIb reports the budget in terms of simulated hours (for one set of customer requests) for All, IncrTime, and $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$. For $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$, we also report in parentheses the number of corresponding generations.

Implementation: For all the approaches, as search algorithm we adopted the genetic algorithm NSGA-II [15], using the jMetalPy framework [9]. We used the default settings of NSGA-II in jMetalPy, as literature reports that default settings can provide reasonable results [4]: parents selection with Binary tournament, SBX crossover operator, polynomial mutation operator, crossover rate of 0.9, and mutation rate equal to the reciprocal of the number of variables.

B. Running of the Experiments and Evaluation Metrics

An *experiment* is the execution of one approach (see Sect. IV-A) using one setting of $NumGens$, D^{SI}, $NumSRs$, and $NumReqs$. Since the search algorithm is affected by randomness, we executed each experiment 30 times as suggested by Arcuri and Briand [3] in their guideline on running experiments with randomized algorithms.

Since the compared approaches are multi-objective, they provide as output a Pareto front of solutions. A classical

approach to assess the quality of a Pareto front is to use some quality indicator [22], that assigns a numerical value to it. By following the guideline of Ali et al. [1], we select IGD as quality indicator, as it provides the most representative results when evaluating NSGA-II. Note that the lower IGD, the better.

Following the guideline of Arcuri and Briand [3], we compare two approaches APP1 and APP2, by comparing the distributions of their 30 values of IGD using suitable statistical tests. The Mann-Whitney U test determines whether there is a significant difference among the distributions[4]; the null hypothesis is that there is no significant difference, and if the p-value returned by the test is lower than the confidence value α ($\alpha = 0.05$ in this work), then we reject the null hypothesis. In case of significant difference, we use the Vargha and Delaney's $\hat{A}_{12}$ effect size [28] to assess the strength of the significance; if $\hat{A}_{12}$ is lower than 0.5, the results of APP1 are significantly lower (and so better in our case) than those of the second approach. By following Kitchenham et al. [20], we identify the following categories of strength: *negligible* when $\hat{A}_{12} \in (0.494, 0.5)$, *small* when $\hat{A}_{12} \in (0.362, 0.494]$, *medium* when $\hat{A}_{12} \in (0.286, 0.362]$, and *large* when $\hat{A}_{12} \leq 0.286$. Similar categories can be identified for $\hat{A}_{12}$ higher than 0.5, i.e., when APP1 is significantly worse.

C. Research questions

We evaluate the experimental results using the following research questions.

RQ1: Do the proposed incremental approaches `IncrSim` and `IncrTime` produce solutions that are comparable to those of the baseline `All`?

RQ2: How do the proposed incremental approaches `IncrSim` and `IncrTime` compare with $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$ and $\texttt{All}^{\texttt{time}}_{\texttt{SB}}$, i.e., the baseline approach executed for the same budget?

RQ3: How do the proposed incremental approaches `IncrSim` and `IncrTime` compare with each other?

We will answer RQ1 and RQ2 by considering the two allocation settings `PartAllo` and `FullAllo` separately; RQ3, instead, can only be answered in the `FullAllo` setting.

V. EXPERIMENT RESULTS

Table IV reports the results for the `PartAllo` setting. Specifically, for all the configurations of the search problems in terms of D^{SI}, *NumGens*, *NumSRs*, and *NumReqs*, it reports the results of the statistical comparison (see Sect. IV-B) between the incremental approach `IncrSim` and the baseline approaches `All` and $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$. The symbol used for the comparison indicates the magnitude of the effect size (see Sect. IV-B) as explained in the caption.

Table V reports the results for the `FullAllo` setting. In this case, the incremental approaches `IncrTime` and `IncrSim` are compared among themselves, each of them with the baseline `All`, and with their corresponding baseline $\texttt{All}^{\texttt{time}}_{\texttt{SB}}$ and $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$ that use the same budget.

[4]We adopted a non-parametric test as the distributions do not follow a normal distribution (as confirmed by the Shapiro–Wilk test).

TABLE IV: RQ1 and RQ2 – Setting `PartAllo` (Legend. $\equiv$ means that there is no significant difference between the two approaches. ✓ means that the approach on the row is *better* than the approach on the column, while ✗ means that it is worse; the number of symbols determines the strength: *negligible* (✓, ✗), *small* (✓✓, ✗✗), *medium* (✓✓✓, ✗✗✗), and *large* (✓✓✓✓, ✗✗✗✗))

(a) D^{SI} = 5 hours

NumGens	*NumSRs*	*NumReqs*	Statistical comparison	
			`IncrSim` vs. $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$	`IncrSim` vs. `All`
10	5	50	✓✓✓✓	$\equiv$
10	5	100	✓✓✓✓	$\equiv$
10	10	50	✓✓✓	$\equiv$
10	10	100	✓✓✓✓	$\equiv$
20	5	50	$\equiv$	$\equiv$
20	5	100	✓✓	$\equiv$
20	10	50	$\equiv$	$\equiv$
20	10	100	$\equiv$	$\equiv$

(b) D^{SI} = 12 hours

NumGens	*NumSRs*	*NumReqs*	Statistical comparison	
			`IncrSim` vs. $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$	`IncrSim` vs. `All`
10	5	50	✓✓✓	✗✗✗
10	5	100	✓✓✓✓	$\equiv$
10	10	50	✓✓✓✓	$\equiv$
10	10	100	✓✓✓✓	$\equiv$
20	5	50	$\equiv$	$\equiv$
20	5	100	✓✓✓✓	$\equiv$
20	10	50	✓✓✓✓	$\equiv$
20	10	100	✓✓✓	$\equiv$

TABLE V: RQ1, RQ2, and RQ3 – Setting `FullAllo` (Legend as in Table IV)

(a) D^{SI} = 5 hours

NumGens	*NumSRs*	*NumReqs*	Statistical comparison				
			`IncrTime` vs. `IncrSim`	`IncrTime` vs. $\texttt{All}^{\texttt{time}}_{\texttt{SB}}$	`IncrSim` vs. $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$	`IncrTime` vs. `All`	`IncrSim` vs. `All`
10	5	50	$\equiv$	✓✓	$\equiv$	$\equiv$	$\equiv$
10	5	100	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$
10	10	50	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$
10	10	100	$\equiv$	✓✓	$\equiv$	$\equiv$	$\equiv$
20	5	50	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$
20	5	100	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$
20	10	50	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$
20	10	100	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$

(b) D^{SI} = 12 hours

NumGens	*NumSRs*	*NumReqs*	Statistical comparison				
			`IncrTime` vs. `IncrSim`	`IncrTime` vs. $\texttt{All}^{\texttt{time}}_{\texttt{SB}}$	`IncrSim` vs. $\texttt{All}^{\texttt{sim}}_{\texttt{SB}}$	`IncrTime` vs. `All`	`IncrSim` vs. `All`
10	5	50	✓✓✓	✓✓✓✓	$\equiv$	✓✓✓	$\equiv$
10	5	100	✓✓	✓✓	$\equiv$	$\equiv$	$\equiv$
10	10	50	✓✓✓	✓✓✓	$\equiv$	$\equiv$	$\equiv$
10	10	100	$\equiv$	✓✓✓	$\equiv$	$\equiv$	$\equiv$
20	5	50	$\equiv$	✓✓✓	$\equiv$	$\equiv$	$\equiv$
20	5	100	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$
20	10	50	$\equiv$	✓✓✓	$\equiv$	$\equiv$	$\equiv$
20	10	100	$\equiv$	$\equiv$	$\equiv$	$\equiv$	$\equiv$

A. RQ1

In this RQ, we want to investigate whether the incremental approaches can obtain results similar to those obtained by the costly approach `All` that always run all the simulations for the whole service interval. As explained before, we will analyse the results of the two allocation settings separately.

1) `PartAllo` setting: In this setting, only the incremental approach `IncrSim` is applicable. From Table IV, we observe

that IncrSim and All are always equivalent, except for one setting. This shows that the heuristic applied by the incremental approach is effective and can find the same optimal solutions of the All in much less time.

2) FullAllo setting: In this setting, both IncrSim and IncrTime are applicable. From Table V, we observe that IncrSim is always equivalent to the All approach. This confirms the effectiveness of considering sets of requests in an incremental fashion.

Regarding IncrTime, we also observe that it is almost always equivalent to the All approach, and in one setting even significantly better. This confirms that also considering the duration of the service delivery in an incremental fashion pays off, i.e., it does not diminish the quality of the solutions and the search is faster.

Answer to RQ1: The incremental approaches IncrSim and IncrTime allow to find solutions as good as those of the costly baseline using much less budget.

B. RQ2

In this RQ, we are interested in evaluating whether IncrSim and IncrTime achieve better performance than the baseline approach when it is given the same budget, in terms of number of simulations or simulated hours (approaches $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ and $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$).

1) PartAllo setting: In this setting, we can only compare IncrSim with $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$. Recall that $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ uses the same (or slightly higher) number of simulations of IncrSim; since it always simulates an individual for all the sets of customer requests, it is run for less generations than IncrSim, as explained in Sect. IV-A.

From Table IV, we observe that IncrSim is almost always better. This shows that, given a limited budget, it is better to allocate the available simulations in an incremental fashion for more generations (possibly having imprecise results in earlier generations), rather than doing less generations with more precise results. $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ becomes equivalent to IncrSim only for $D^{SI} = 5$ (which is an easier problem) when the number of generations $NumGens$ for IncrSim is 20 (which corresponds to 12 generations for $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$).

2) FullAllo setting: From Table V, we observe that IncrSim is always equivalent to $\texttt{All}_{\texttt{SB}}^{\texttt{sim}}$ in the FullAllo setting. Note that the FullAllo problem is much simpler than the PartAllo problem (i.e., the search space is much smaller), and so it is possible that the higher number of generations guaranteed by IncrSim are not necessary.

However, by looking at the comparison of IncrTime with $\texttt{All}_{\texttt{SB}}^{\texttt{time}}$, we observe that, while they are almost always equivalent for $D^{SI} = 5$, IncrTime is almost always significantly better for $D^{SI} = 12$. This shows that IncrTime has the potential of being a very effective approach, as it is able to improve the results even in this simple setting, for which IncrSim, instead, does not provide any advantage.

Answer to RQ2: Overall, allocating the budget in an incremental fashion for more generations using an approximate fitness (as done by IncrSim and IncrTime), provides better results than using fewer generations with a more precise fitness.

C. RQ3

In this RQ, we are interested in investigating which of the two incremental approaches IncrSim and IncrTime is better. The comparison can be done only in the FullAllo setting, as it is the only one in which IncrTime is applicable. From Table V, we observe that, while there is no difference for $D^{SI} = 5$, for three settings of $D^{SI} = 12$ IncrTime is better. This, together with the results of RQ2 for IncrTime, seems to confirm that IncrTime is a potentially good approach. As future work, we plan to devise techniques to apply IncrTime also in the more complicated setting PartAllo, to see if it is advantageous also in that case.

Answer to RQ3: Overall, there is no significant difference between IncrSim and IncrTime, although IncrTime can obtain better results in few settings.

VI. Threats to validity

The validity of the proposed approaches could be affected by some threats. We discuss them, following the classical classification of *construct*, *conclusion*, *internal*, and *external validities* [29].

Construct validity. A construct validity threat could be that the metrics that we are using in the assessment of the experiments do not reflect the object of the investigation. Since the goal of the approach is to find different allocations that show the tradeoff among the three concerns (see Sect. II-A), using a quality indicator as IGD is appropriate. Moreover, given the availability of several quality indicators [22], we selected the one that has been shown to be the most representative of different other indicators when assessing solutions produced by NSGA-II [1].

Conclusion validity. A threat of this type is that the random behaviour of the adopted search algorithm may not allow to draw definitive conclusions. So, by following the guideline of Arcuri and Briand [3], we executed each experiment 30 times and compared the results of two approaches using the Mann-Whitney U test and $\hat{A}_{12}$ statistics.

Internal validity. A typical threat of this type is that the relationship between the applied approach and the obtained results is by chance. For example, an *instrumentation* threat in the experiments execution [29] could be that the results are due to a faulty implementation. To mitigate this threat, we have carefully checked the implementation; specifically, we have tested that the incremental approach IncrSim only considers some of the sets of customer requests, and that IncrTime only considers part of the simulation.

External validity. One threat of this type is that the proposed approach may not be generalisable to other contexts. Regarding `IncrSim`, we think that it is, in principle, applicable to different contexts where a system must be assessed against different sets representative input data; however, the effectiveness of the approach would probably depend on the variability of the results across the different sets of data. Regarding `IncrTime`, instead, the approach could be applied only to systems that consume input data in a similar way as done by the considered delivery system. In any case, we want to point out that we agree with Briand et al. [13] that research conducted with industry is, by definition, context-driven since it targets the specific problem of the industrial partner, and applicability to other contexts should not be primary concern. This will not prevent us to investigate whether the proposed approaches can be applied/adapted to other contexts.

VII. Related work

In this paper, we propose a search-based approach for the optimisation of an autonomous system. Therefore, in Sect. VII-A, we review related works on search-based approaches for autonomous systems. Our work proposes two types of incremental search that have the goal to increase the scalability of the search, which is affected by the costly execution of the simulator; therefore, in Sect. VII-B we review works that propose techniques to achieve scalability in search.

A. Search-based approaches for autonomous systems

Autonomous systems are complex systems operating in infinite and multi-dimensional environments, which makes their optimisation and validation very challenging. In software engineering, meta-heuristic search has been proposed as a solution for different problems [18].

In the field of autonomous systems, search-based testing (SBT) has been largely applied to autonomous and automated driving systems (ADSs).

For example, Ben Abdessalem et al. [7] propose an SBT approach to test an Automated Emergency Braking (AEB), that is an ADS component to apply emergency braking in case of detection of a too-close object; specifically, they search for critical test scenarios in which the autonomous vehicle collides with a pedestrian who is trying to cross the street, showing failures of the AEB. Gambi et al. [17] combine *procedural content generation* (a technique used in video games) and genetic algorithms to generate roads where the lane-keeping component of an ADS fails in keeping the vehicle on the road.

Li et al. [21] propose an SBT approach combined with fuzzing (AV-FUZZER), that tries to find safety violations of an ADS implemented in the Apollo framework; specifically, the goal of AV-FUZZER is to find scenarios violating a *safety potential* metric, which determines whether a vehicle, in case of danger, can stop without colliding.

Luo et al. [24] proposed an approach that can efficiently generate scenarios covering different types of *patterns* of satisfaction and violation of the different ADS requirements, such as guaranteeing safety, complying to the traffic regulations, etc.

Ben Abdessalem et al. [8] propose a search approach to find safety violations due to feature interactions. Indeed, the different components of an ADS (*features*)—e.g., the Adaptive Cruise Control (ACC) or the Automated Emergency Braking (AEB)— although they may work correctly individually, they could lead to failures when working together, i.e., a feature could override the command provided by another feature. The authors propose FITEST (Feature Interaction TESTing), an SBT approach to discover harmful feature interactions.

Search-based approaches have been proposed also for other ADS testing phases, such as test case prioritisation. Birchler et al. [11] propose SDC-Prioritizer, which finds tests to prioritise in ADS testing only relying on the static features of the roads. Lu et al. [23] propose SPECTRE, a multi-objective search-based approach for prioritisation of test scenarios to test newer ADS versions, based on four optimisation objectives.

All previous approaches consider autonomous systems different from the one considered in this work. However, our incremental approaches could still be applicable if their fitness functions relay on multiple simulations.

B. Techniques for achieving scalability in search

Scalability of search-based approaches is a relevant issue in domains in which the fitness computation is expensive. A common approach to tackle this issue is to use *surrogate models* that approximate the fitness functions and can replace them, so to avoid the cost of the fitness computation. Please refer to Tong et al. [27], Santana-Quintero et al. [26], and Jin [19] for extensive surveys on surrogate models for search.

For example, Ben Abdessalem et al. [6] use surrogate models to speed up the testing of a Pedestrian Detection Vision (PeVi) system, that can be simulated with a costly simulator; the approach uses the surrogate model to get an approximation of the fitness value and, only if needed, calls the simulator to have more precise values. The approach shares with our approaches the idea of avoiding expensive simulations; however, our optimisation is based on the position of the generation during the search, while the approach in [6] depends on the specific individual (regardless of the generation).

Menghi et al. [25] test models of cyber-physical systems (CPS) using *falsification*, a type of search-based testing that tries to violate requirement specified in Signal Temporal Logic. Since some CPS models can be very expensive to simulate, they propose ARIsTEO, a falsification approach that approximates the CPS model with a surrogate model obtained through the *system identification* approach.

Bentley et al. [10] recently considered a delivery system inspired by the one used in this work. In order to speed up the search for the allocation problem, they use variational autoencoders that learn valid regions of the search space.

VIII. Conclusions

In this paper, we proposed a search-based approach to generate different configurations for an automatic delivery service based on autonomous robots. The approach generates different solutions showing the trade-off existing between the

cost of providing the service, the quality of the service (i.e., the amount of served requests), and the level of safety of the service. The approach relies on costly fitness functions that need to simulate the service multiple times using different sets of customer requests; so, we proposed an improvement (`IncrSim`) that avoids simulating all the sets of requests for all the individuals, and another improvement (`IncrTime`) that avoids running the whole simulation.

Experiments show that both `IncrSim` and `IncrTime` are effective. However, we could only apply `IncrTime` in the simpler problem (`FullAllo`) in which the robots are executed all the time. Since `IncrTime` provided better results than `IncrSim` in `FullAllo`, as future work we plan to devise techniques to apply it also in the more complex setting `PartAllo`. Moreover, we currently apply the two heuristics separately; as future work, we plan to assess whether they can be effectively applied together. Finally, the heuristics in this work are applied considering the generation of the search; as future work, we plan to assess whether they could be applied considering the specific individual, e.g., avoiding running several simulations for individuals that appear to have low performance on the first simulations.

Our incremental approaches use NSGA-II as underling search algorithm, but any population-based search algorithm could be used. As future work, we plan to assess the proposed approaches using different search algorithms.

REFERENCES

[1] S. Ali, P. Arcaini, D. Pradhan, S. A. Safdar, and T. Yue. Quality indicators in search-based software engineering: An empirical evaluation. *ACM Trans. Softw. Eng. Methodol.*, 29(2), Mar. 2020.

[2] P. Arcaini, E. Castellano, F. Ishikawa, H. Kawamoto, K. Sawai, and E. Muramoto. Supplementary material for the paper "Incremental Search-Based Allocation of Autonomous Robots for Goods Delivery". https://github.com/ERATOMMSD/allocation-delivery-robots, 2023.

[3] A. Arcuri and L. Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering*, ICSE '11, pages 1–10, New York, NY, USA, 2011. ACM.

[4] A. Arcuri and G. Fraser. Parameter tuning or default values? an empirical investigation in search-based software engineering. *Empirical Software Engineering*, 18:594–623, 2013.

[5] I. Bakach, A. M. Campbell, and J. F. Ehmke. Robot-based last-mile deliveries with pedestrian zones. *Frontiers in Future Transportation*, 2, 2022.

[6] R. Ben Abdessalem, S. Nejati, L. C. Briand, and T. Stifter. Testing advanced driver assistance systems using multi-objective search and neural networks. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ASE 2016, pages 63–74, New York, NY, USA, 2016. ACM.

[7] R. Ben Abdessalem, S. Nejati, L. C. Briand, and T. Stifter. Testing vision-based control systems using learnable evolutionary algorithms. In *Proceedings of the 40th International Conference on Software Engineering*, ICSE '18, pages 1016–1026, New York, NY, USA, 2018. ACM.

[8] R. Ben Abdessalem, A. Panichella, S. Nejati, L. C. Briand, and T. Stifter. Testing autonomous cars for feature interaction failures using many-objective search. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, ASE 2018, pages 143–154, New York, NY, USA, 2018. ACM.

[9] A. Benítez-Hidalgo, A. J. Nebro, J. García-Nieto, I. Oregi, and J. Del Ser. jMetalPy: A Python framework for multi-objective optimization with metaheuristics. *Swarm and Evolutionary Computation*, 51:100598, 2019.

[10] P. J. Bentley, S. L. Lim, P. Arcaini, and F. Ishikawa. Using a variational autoencoder to learn valid search spaces of safely monitored autonomous robots for last-mile delivery. In *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '23, New York, NY, USA, 2023. Association for Computing Machinery.

[11] C. Birchler, S. Khatiri, P. Derakhshanfar, S. Panichella, and A. Panichella. Single and multi-objective test cases prioritization for self-driving cars in virtual environments. *ACM Trans. Softw. Eng. Methodol.*, may 2022.

[12] K. Braekers, K. Ramaekers, and I. Van Nieuwenhuyse. The vehicle routing problem: State of the art classification and review. *Computers & Industrial Engineering*, 99:300–313, 2016.

[13] L. C. Briand, D. Bianculli, S. Nejati, F. Pastore, and M. Sabetzadeh. The case for context-driven software engineering research: Generalizability is overrated. *IEEE Software*, 34(5):72–75, 2017.

[14] C. Chen, E. Demir, Y. Huang, and R. Qiu. The adoption of self-driving delivery robots in last mile logistics. *Transportation Research Part E: Logistics and Transportation Review*, 146:102214, 2021.

[15] K. Deb, A. Pratap, S. Agarwal, and T. A. M. T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[16] M. Figliozzi and D. Jennings. Autonomous delivery robots and their potential impacts on urban freight energy consumption and emissions. *Transportation Research Procedia*, 46:21–28, 2020.

[17] A. Gambi, M. Mueller, and G. Fraser. Automatically testing self-driving cars with search-based procedural content generation. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2019, pages 318–328, New York, NY, USA, 2019. Association for Computing Machinery.

[18] M. Harman, S. A. Mansouri, and Y. Zhang. Search-based software engineering: Trends, techniques and applications. *ACM Comput. Surv.*, 45(1), Dec. 2012.

[19] Y. Jin. Surrogate-assisted evolutionary computation: Recent advances and future challenges. *Swarm and Evolutionary Computation*, 1(2):61–70, 2011.

[20] B. Kitchenham, L. Madeyski, D. Budgen, J. Keung, P. Brereton, S. Charters, S. Gibbs, and A. Pohthong. Robust statistical methods for empirical software engineering. *Empirical Softw. Engg.*, 22(2):579–630, apr 2017.

[21] G. Li, Y. Li, S. Jha, T. Tsai, M. B. Sullivan, S. K. S. Hari, Z. Kalbarczyk, and R. K. Iyer. AV-FUZZER: Finding safety violations in autonomous driving systems. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, pages 25–36, 2020.

[22] M. Li and X. Yao. Quality evaluation of solution sets in multiobjective optimisation: A survey. *ACM Comput. Surv.*, 52(2), Mar. 2019.

[23] C. Lu, H. Zhang, T. Yue, and S. Ali. Search-based selection and prioritization of test scenarios for autonomous driving systems. In *Search-Based Software Engineering: 13th International Symposium, SSBSE 2021, Bari, Italy, October 11–12, 2021, Proceedings*, pages 41–55, Berlin, Heidelberg, 2021. Springer-Verlag.

[24] Y. Luo, X.-Y. Zhang, P. Arcaini, Z. Jin, H. Zhao, F. Ishikawa, R. Wu, and T. Xie. Targeting requirements violations of autonomous driving systems by dynamic evolutionary search. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 279–291, 2021.

[25] C. Menghi, S. Nejati, L. Briand, and Y. I. Parache. Approximation-refinement testing of compute-intensive cyber-physical models: An approach based on system identification. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, pages 372–384, New York, NY, USA, 2020. Association for Computing Machinery.

[26] L. V. Santana-Quintero, A. A. Montaño, and C. A. C. Coello. *A Review of Techniques for Handling Expensive Functions in Evolutionary Multi-Objective Optimization*, pages 29–59. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.

[27] H. Tong, C. Huang, L. L. Minku, and X. Yao. Surrogate models in evolutionary single-objective optimization: A new taxonomy and experimental study. *Information Sciences*, 562:414–437, 2021.

[28] A. Vargha and H. D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.

[29] C. Wohlin, P. Runeson, M. Hst, M. C. Ohlsson, B. Regnell, and A. Wessln. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated, 2012.